

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

TECHNICAL UNIVERSITY OF MOLDOVA

FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
DEPARTMENT OF SOFTWARE AND AUTOMATION ENGINEERING

# Report

*Laboratory Work No. 12*

## The bounded buffer producer-consumer problem, condition variables, readers-writers synchronization, and deadlock prevention in Linux

Elaborated by:
st. gr. FAF-241
**Tihon Aurelian-Mihai**

Verified by:
univ. assist.
**Reiţman Patricia**

Chişinău – 2026

# Contents

**INTRODUCTION** **1**

**1 THEORETICAL BACKGROUND** **1**

1.1 The bounded buffer producer-consumer architecture . . . . . 2

1.2 Condition variables and monitor semantics . . . . . 2

1.3 Readers-writers coordination and starvation dynamics . . . . . 3

1.4 Resource allocation graphs and Coffman deadlock conditions . . . . . 3

**2 THE BOUNDED BUFFER PRODUCER-CONSUMER PROBLEM** **4**

2.1 Implementation and buffer tracking architecture . . . . . 5

2.2 Analysis of inverted acquisition order and nested monitor deadlock . . . . . 6

**3 CONDITION VARIABLES AND MONITOR SYNCHRONIZATION** **6**

3.1 State-predicated signaling via condition variables . . . . . 7

3.2 Evaluation of predicate loop guards versus conditional branching . . . . . 7

**4 READERS-WRITERS SYNCHRONIZATION AND CONCURRENCY** **8**

4.1 Reader preference coordination and entry protocols . . . . . 9

4.2 Counter protection necessity and writer starvation . . . . . 10

**5 DEADLOCK MODELING AND ORDERED ACQUISITION PREVENTION** **10**

5.1 Empirical deadlock demonstration and timeout monitoring . . . . . 11

5.2 Heisenbug analysis of uncoordinated execution . . . . . 11

5.3 Mathematical prevention via globally consistent lock ordering . . . . . 12

**6 CONCLUSION** **13**

**REFERENCES** **14**

**ANNEX A: COMPLETE SOURCE CODE LISTINGS** **15**

# INTRODUCTION

Multi-threaded concurrent systems require sophisticated synchronization patterns to manage cooperative resource sharing, prevent data corruption, and avoid catastrophic deadlock states. While fundamental mutual exclusion primitives such as binary mutexes guarantee that critical sections execute serially, practical software engineering problems routinely involve asymmetric producer-consumer dependencies, state-predicated wait conditions, differential reader-writer access controls, and multi-resource contention.

This laboratory work investigates four foundational multi-threaded synchronization paradigms implemented in the C programming language within the Linux operating system environment utilizing the POSIX Threads (`pthreads`) and semaphore libraries. The investigation proceeds across four sequential synchronization domains: first, coordinating asymmetric rates between producing and consuming threads using dual counting semaphores and a mutual exclusion lock guarding a circular queue; second, implementing execution ordering through explicit condition variables and evaluating the necessity of predicate verification loops; third, enforcing concurrent reader access while guaranteeing strict mutual exclusion for writers; and fourth, demonstrating circular wait deadlocks under inverted mutex acquisition orders and proving their elimination via globally consistent lock ordering.

All implementations were evaluated under native Linux execution environments, and timing traces were systematically gathered to verify mathematical correctness and deadlock-free operation.

# 1 THEORETICAL BACKGROUND

## 1.1 The bounded buffer producer-consumer architecture

The bounded buffer (or bounded queue) is a quintessential synchronization archetype in systems programming, underpinning kernel mechanisms such as Unix pipes, socket buffers, and device driver queues. A producer thread generates discrete data units and places them into a shared buffer of fixed capacity $K$, while a consumer thread removes items from the buffer for downstream processing.

To maintain buffer integrity and eliminate race conditions, three distinct synchronization constraints must be satisfied simultaneously:

$$\text{Capacity Invariant:} \quad 0 \leq \mathscr{N}_{\text{items}} \leq K \tag{1.1}$$

$$\text{Mutual Exclusion:} \quad |\mathscr{T}_{\text{active}}| \leq 1 \quad \forall \text{ operations on buffer pointers} \tag{1.2}$$

$$\text{Absence of Busy-Waiting:} \quad \text{Threads sleep when } \mathscr{N}_{\text{items}} = K \text{ or } \mathscr{N}_{\text{items}} = 0 \tag{1.3}$$

This tripartite model is implemented through two counting semaphores and one mutex. Specifically, $\mathscr{S}_{\text{empty}}$ is initialized to capacity $K$, tracking the number of currently available empty buffer slots, which the producer decrements prior to inserting an item. In parallel, $\mathscr{S}_{\text{full}}$ is initialized to zero, tracking unconsumed items, which the consumer decrements prior to extracting data. Finally, $\mathscr{M}_{\text{lock}}$ provides mutual exclusion serializing updates to buffer insertion and removal indices and the active item counter.

A critical architectural pitfall in bounded buffer implementations involves the sequential nesting of lock acquisitions. If a thread acquires $\mathscr{M}_{\text{lock}}$ before calling `sem_wait(&empty)`, a full buffer will cause the producer to sleep while holding the buffer lock. Consequently, the consumer cannot acquire $\mathscr{M}_{\text{lock}}$ to remove items, inducing a fatal nested monitor deadlock.

## 1.2 Condition variables and monitor semantics

A condition variable (`pthread_cond_t`) provides a mechanism for threads to suspend execution until a specific shared data state predicate $\mathscr{P}$ evaluates to true. Unlike semaphores, condition variables are entirely stateless: they do not maintain internal counters or history. If a signal is emitted when no thread is actively waiting on the condition variable, the signal is permanently discarded.

The fundamental contract of `pthread_cond_wait(&cond, &mutex)` dictates three atomic transitions:

$$\text{pthread_cond_wait}(\mathscr{C}, \mathscr{M}): \quad \text{release}(\mathscr{M}) \longrightarrow \text{enqueue_sleep}(\mathscr{C}) \longrightarrow \text{acquire}(\mathscr{M}) \tag{1.4}$$

When awakened by signaling or broadcasting the condition variable, the waiting thread must re-acquire $\mathscr{M}$ before exiting the wait primitive.

Under modern operating systems utilizing Mesa-style monitor semantics (standardized by POSIX), the signaling thread wakes a waiting thread while holding the lock. By the time the awakened thread re-acquires the mutex, another thread may have acquired the lock and invalidated the predicate $\mathscr{P}$. Furthermore, POSIX explicitly permits *spurious wakeups*, wherein `pthread_cond_wait` returns without any explicit signal due to internal kernel context switches or hardware interrupt handling. Consequently, conditional waits must strictly be encapsulated within a predicate verification loop:

```
pthread_mutex_lock(&lock);
while (!predicate) {
    pthread_cond_wait(&changed, &lock);
}
// Predicate is guaranteed true here
pthread_mutex_unlock(&lock);
```

### 1.3 Readers-writers coordination and starvation dynamics

The readers-writers problem models access to shared data where read operations are non-destructive and may proceed concurrently, whereas write operations modify data and require exclusive access. The standard first readers-writers solution employs two binary semaphores: $\mathscr{S}_{\text{room}}$, initialized to 1, which represents exclusive access to the shared room for writers, and $\mathscr{S}_{\text{rlock}}$, initialized to 1, which protects concurrent updates to the active reader counter $R$.

The entry protocol for an arriving reader increments $R$; if $R = 1$ (the reader is the first to arrive), it executes `sem_wait(&room)`, effectively barring all writers from entering. Subsequent readers increment $R$ and bypass $\mathscr{S}_{\text{room}}$ entirely. Upon departure, a reader decrements $R$; if $R = 0$ (the departing reader is the last to leave), it executes `sem_post(&room)`, reopening access to waiting writers.

While this protocol guarantees maximal reader concurrency, it introduces the pathology of writer starvation (indefinite postponement). If a continuous stream of reader threads arrives such that $R > 0$ indefinitely, waiting writer threads remain blocked indefinitely, never gaining access to the resource.

### 1.4 Resource allocation graphs and Coffman deadlock conditions

A deadlock represents a system state wherein every thread in a set $\mathscr{T} = \{T_1, T_2, \ldots, T_n\}$ is blocked waiting for an event that can only be triggered by another thread within $\mathscr{T}$. According to Coffman et al., a deadlock can occur if and only if four structural conditions hold simultaneously. First, the mutual exclusion

condition requires that resources cannot be shared simultaneously, such that each lock is held by at most one thread. Second, the hold and wait condition permits a thread currently holding allocated resources to request additional ones. Third, the no preemption condition dictates that allocated resources cannot be forcibly revoked by the kernel. Fourth, the circular wait condition establishes a closed dependency cycle wherein each thread waits for a resource held by the subsequent thread in the chain.

Deadlock prevention strategies systematically invalidate at least one of these four conditions. The most robust and computationally efficient software engineering technique is the enforcement of a global partial order $\prec$ on all lock acquisitions. If every thread acquires resources in strictly ascending order according to $\prec$, circular wait cycles cannot form, rendering deadlock mathematically impossible.

# 2 THE BOUNDED BUFFER PRODUCER-CONSUMER PROBLEM

## 2.1 Implementation and buffer tracking architecture

To evaluate cooperative rate regulation across shared memory, the program `prodcons.c` was constructed. A circular buffer of capacity $K = 4$ slots (`buf[SIZE]`) is accessed concurrently by a high-speed producer thread and an intentionally decelerated consumer thread (`usleep(300000)`).

The synchronization protocol coordinates state across iterations:

```
// Producer routine:
sem_wait(&empty);                // Wait for an available empty slot
pthread_mutex_lock(&lock);       // Isolate buffer index updates
buf[in] = item;
in = (in + 1) % SIZE;
count++;
printf("put %d (in buffer: %d)\n", item, count);
pthread_mutex_unlock(&lock);
sem_post(&full);                 // Signal availability of a populated slot

// Consumer routine:
sem_wait(&full);                 // Wait for an available populated slot
pthread_mutex_lock(&lock);       // Isolate buffer extraction
int item = buf[out];
out = (out + 1) % SIZE;
count--;
printf(" take %d (in buffer: %d)\n", item, count);
pthread_mutex_unlock(&lock);
sem_post(&empty);                // Signal availability of a freed slot
usleep(300000);                  // Emulate slow consumer processing
```

As demonstrated by the live execution trace illustrated in Figure 2.1, although the producer operates with zero artificial delay, the total populated buffer count strictly satisfies $0 \leq \texttt{count} \leq 4$.

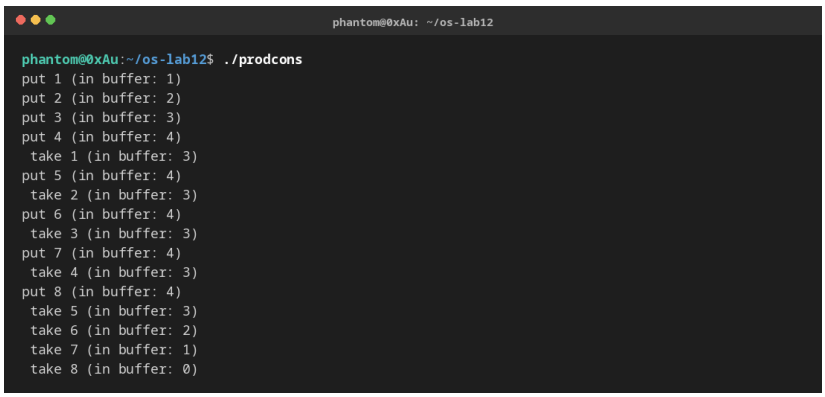



**Figure 2.1 – Empirical execution trace of the bounded buffer producer-consumer implementation.**

The trace confirms that the producer immediately deposits items 1 through 4, at which point $\mathscr{S}_{\text{empty}}$ reaches zero. The subsequent call `sem_wait(&empty)` on item 5 suspends the producer thread until the slow consumer removes item 1, releasing an empty slot via `sem_post(&empty)`. Conversely, at initial program launch, the consumer thread starts first but immediately blocks on `sem_wait(&full)` because $\mathscr{S}_{\text{full}}$ is initialized to 0. It remains blocked until the producer deposits the first item and signals completion.

## 2.2 Analysis of inverted acquisition order and nested monitor deadlock

To observe the consequences of improper lock sequencing, the initial two statements of `producer()` were experimentally inverted:

```
1 pthread_mutex_lock(&lock);
2 sem_wait(&empty);
```

Under this inverted sequence, the producer successfully inserts 4 items. On the fifth iteration, the producer acquires `lock` and then invokes `sem_wait(&empty)`. Because the buffer is completely full, $\mathscr{S}_{\text{empty}} = 0$, and the kernel suspends the producer. However, because the producer went to sleep while still holding `lock`, the consumer thread cannot enter the critical section to extract an item: the consumer blocks indefinitely on `pthread_mutex_lock(&lock)`. Both threads remain permanently asleep, creating an unresolvable nested deadlock requiring external process termination.

# 3 CONDITION VARIABLES AND MONITOR SYNCHRONIZATION

## 3.1 State-predicated signaling via condition variables

The program `condvar.c` demonstrates event coordination between writer thread A and reader thread B without relying on the internal counter of a semaphore. In this model, the state of the system is explicitly stored in an ordinary global integer variable:

```
int ready = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t changed = PTHREAD_COND_INITIALIZER;
```

Reader thread B acquires `lock` and verifies whether `ready` is non-zero. If `ready == 0`, thread B invokes `pthread_cond_wait(&changed, &lock)`, which atomically yields the mutex and places thread B into the condition variable wait queue. Meanwhile, writer thread A sleeps for 1 second and acquires `lock`. It writes `"hello"` to `message`, sets `ready = 1`, signals the condition variable via `pthread_cond_signal()`, and releases `lock`, producing the execution trace illustrated in Figure 3.1.

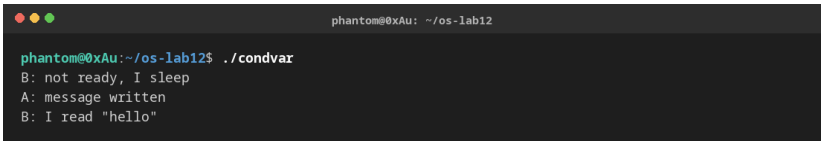



**Figure 3.1 – Event sequencing and condition variable signaling trace of `condvar`.**

The trace confirms that reader thread B executes first, verifies the state condition, prints `"B: not ready, I sleep"`, and suspends. Exactly 1 second later, writer thread A completes the write operation, signals the condition variable, and reader thread B resumes, printing `"B: I read \"hello\""`.

## 3.2 Evaluation of predicate loop guards versus conditional branching

In `condvar.c`, the reader thread encapsulates the condition wait within a `while` loop:

```
while (!ready) {
    pthread_cond_wait(&changed, &lock);
}
```

When this construct was experimentally replaced with a simple conditional branch:

```
if (!ready) {
    pthread_cond_wait(&changed, &lock);
}
```

The program executed without observable errors because the test harness comprised exactly one writer and one reader, with zero intervening state changes.

However, in multi-threaded production architectures, utilizing `if` instead of `while` is fundamentally flawed for two critical reasons. First, operating system kernels, including the Linux futex subsystem, permit `pthread_cond_wait` to return unexpectedly without an explicit signal due to internal signal deliveries or multi-core race resolutions. If an `if` statement is used, a spuriously awakened thread proceeds into the critical section with `ready == 0`, operating on invalid or uninitialized data. Second, in architectures featuring multiple consumer threads waiting on the same condition variable, signaling wakes one or all consumers. The first awakened thread to acquire `lock` consumes the payload and resets `ready = 0`. If subsequent awakened threads bypass predicate re-evaluation, they execute against depleted state buffers. The `while` loop guarantees that awakened threads immediately re-evaluate the state predicate before proceeding.

# 4 READERS-WRITERS SYNCHRONIZATION AND CONCURRENCY

## 4.1 Reader preference coordination and entry protocols

The program `readers.c` implements the classic first readers-writers synchronization pattern, permitting arbitrary concurrent reader threads while guaranteeing that writer threads maintain strict, solitary mutual exclusion. The coordination mechanism relies on two semaphores:

```
sem_t rlock; // Initialized to 1; guards concurrent mutations of 'readers' counter
sem_t room;  // Initialized to 1; enforces exclusive room occupancy for writers
```

The reader coordination protocol enforces asymmetric access:

```
void *reader(void *arg) {
    sem_wait(&rlock);
    if (++readers == 1)
        sem_wait(&room); // First reader locks the room against all writers
    sem_post(&rlock);

    printf("reader %ld reads %d (readers inside: %d)\n", id, value, readers);
    usleep(200000);

    sem_wait(&rlock);
    if (--readers == 0)
        sem_post(&room); // Last reader releases the room for waiting writers
    sem_post(&rlock);
    return NULL;
}
```

As demonstrated in the empirical execution trace in Figure 4.1, writer 1 enters first and mutates `value` to 10 in total isolation. Subsequently, reader threads 1, 2, and 3 enter concurrently, simultaneously observing `value = 10` while reporting active reader counts ascending from 1 to 3. Only after all three reader threads vacate the room does writer 2 gain access to execute its write operation (`value = 20`).




**Figure 4.1 – Concurrent execution trace of reader threads and solitary writer execution.**

### 4.2 Counter protection necessity and writer starvation

The global integer `readers` tracks the active population of readers. Modifying this counter decomposes into three separate CPU instructions: memory load, register increment/decrement, and memory store. Without `rlock`, concurrent reader threads would induce race conditions. Two readers could simultaneously load `readers = 0`, increment to 1, and both invoke `sem_wait(&room)`, causing an erroneous double lock. Alternatively, concurrent departures could drop decrements, resulting in `readers` never reaching 0 and permanently deadlocking all subsequent writers.

Furthermore, this implementation demonstrates inherent writer starvation. Because incoming readers bypass $\mathscr{S}_{\text{room}}$ while `readers` $> 0$, continuous reader arrival prevents `-readers == 0` from being reached. Consequently, writers remain blocked on `sem_wait(&room)` indefinitely. Resolving this pathology requires second readers-writers solutions (writer-preference) utilizing turnstile semaphores to block new readers once a writer signals intent to write.

# 5 DEADLOCK MODELING AND ORDERED ACQUISITION PREVENTION

## 5.1 Empirical deadlock demonstration and timeout monitoring

To demonstrate the physical occurrence of deadlocks, the program `deadlock.c` was constructed. Two threads concurrently attempt to acquire two mutex locks, `A` and `B`, in reverse order:

$$\text{Thread one:} \quad \text{lock}(A) \longrightarrow \text{sleep}(1) \longrightarrow \text{lock}(B) \tag{5.1}$$

$$\text{Thread two:} \quad \text{lock}(B) \longrightarrow \text{sleep}(1) \longrightarrow \text{lock}(A) \tag{5.2}$$

When executed, thread `one` claims mutex $A$ while thread `two` claims mutex $B$. After the 1-second sleep delay, thread `one` requests mutex $B$ (held by `two`), and thread `two` requests mutex $A$ (held by `one`), locking both threads into an unresolvable circular wait. Figure 5.1 illustrates the terminal output when executing the broken program under a 5-second watchdog timer, followed by the execution of the resolved program.




**Figure 5.1 – Deadlock manifestation under inverted lock ordering and resolution via uniform ordering.**

The captured output demonstrates that the broken implementation hangs indefinitely, causing the watchdog utility `timeout 5` to terminate the process with exit code 124.

## 5.2 Heisenbug analysis of uncoordinated execution

When the artificial delay `sleep(1)` was removed from both threads in the uncoordinated version, ten consecutive executions produced non-deterministic results: six runs completed successfully with exit code 0, while four runs deadlocked with exit code 124.

This non-deterministic manifestation represents a classic *Heisenbug*. Without artificial delays, deadlock only triggers if thread preemption occurs precisely between the first and second lock acquisitions. If one thread executes both lock acquisitions before the other thread is scheduled, the program completes

normally. This behavior is substantially more hazardous in software engineering than a deterministic hang, as the timing vulnerability easily eludes automated test suites and manifests unpredictably under heavy production workloads.

### 5.3 Mathematical prevention via globally consistent lock ordering

To permanently eliminate deadlock, the acquisition sequence in thread `two` was restructured to enforce identical lock acquisition ordering:

```
// Fixed thread two routine:
pthread_mutex_lock(&A); // Acquire A first, identical to thread one
pthread_mutex_lock(&B); // Acquire B second
```

By establishing a total ordering $\mathcal{O}(A) < \mathcal{O}(B)$, all threads request resources along a directed acyclic graph. If thread `one` holds mutex $A$, thread `two` blocks immediately on its initial acquisition of $A$ without acquiring $B$. Consequently, thread `one` is guaranteed unhindered access to acquire $B$, execute its critical section, and release both locks. Circular wait is rendered mathematically impossible, guaranteeing deterministic execution with exit code 0.

# 6 CONCLUSION

This laboratory work conducted an exhaustive experimental evaluation of multi-threaded synchronization primitives in the Linux operating system, addressing the classic challenges of concurrent systems programming. Through the systematic implementation and empirical verification of bounded buffer producer-consumer pipelines, condition variable signaling, reader-writer mutual exclusion, and deadlock resolution, the operational mechanisms governing thread coordination were established.

The investigation demonstrated that counting semaphores provide an elegant, mathematically sound abstraction for bounded capacity regulation. By coordinating asymmetric thread execution rates through dual counting semaphores and a mutual exclusion lock, the bounded buffer maintained strict capacity invariants without CPU polling. Furthermore, empirical testing revealed that inversion of lock acquisition order within nested synchronization structures immediately induces fatal monitor deadlocks, underscoring the absolute necessity of rigorous lock hierarchy design.

The exploration of condition variables established the vital distinction between stateless signaling queues and stateful counter mechanisms. The empirical analysis proved that Mesa-style monitor semantics and kernel-level spurious wakeups mandate the encapsulation of conditional wait primitives within predicate verification loops. In parallel, the readers-writers implementation validated the feasibility of high-concurrency read operations alongside strict write isolation, while simultaneously demonstrating the inherent vulnerability of reader-preference algorithms to writer starvation.

Finally, the empirical modeling of mutex deadlocks verified Coffman's circular wait condition in physical systems. The non-deterministic behavior observed upon removing artificial timing delays highlighted the treacherous nature of concurrency Heisenbugs in distributed and multi-threaded systems. By enforcing a globally consistent partial ordering on lock acquisitions, resource dependency cycles were completely eliminated, providing a verifiable guarantee of deadlock freedom. These core principles form the architectural foundation for robust systems engineering across modern multi-core operating systems.

## REFERENCES

[1] E. G. Coffman, M. J. Elphick, and A. Shoshani, “System deadlocks,” *ACM Computing Surveys*, vol. 3, no. 2, pp. 67–78, 1971.

[2] C. A. R. Hoare, “Monitors: An operating system structuring concept,” *Communications of the ACM*, vol. 17, no. 10, pp. 549–557, 1974.

[3] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: John Wiley & Sons, 2018.

[4] IEEE and The Open Group, *The Open Group Base Specifications Issue 7, 2018 Edition (POSIX.1-2017)*, IEEE Std 1003.1-2017, 2018.

# ANNEX A: COMPLETE SOURCE CODE LISTINGS

## A.1 Source code of the bounded buffer producer-consumer utility (`prodcons.c`)

```c
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE 4
#define ITEMS 8

int buf[SIZE];
int in = 0, out = 0, count = 0;

sem_t empty, full;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *producer(void *arg) {
    (void)arg;
    for (int item = 1; item <= ITEMS; item++) {
        sem_wait(&empty);
        pthread_mutex_lock(&lock);
        buf[in] = item;
        in = (in + 1) % SIZE;
        count++;
        printf("put %d (in buffer: %d)\n", item, count);
        pthread_mutex_unlock(&lock);
        sem_post(&full);
    }
    return NULL;
}

void *consumer(void *arg) {
    (void)arg;
    for (int k = 0; k < ITEMS; k++) {
        sem_wait(&full);
        pthread_mutex_lock(&lock);
        int item = buf[out];
        out = (out + 1) % SIZE;
        count--;
        printf(" take %d (in buffer: %d)\n", item, count);
        pthread_mutex_unlock(&lock);
        sem_post(&empty);
        usleep(300000);
    }
    return NULL;
}

int main(void) {
    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);

    pthread_t p, c;
    pthread_create(&c, NULL, consumer, NULL);
    pthread_create(&p, NULL, producer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
```

```
    return 0;
}
```

### A.2 Source code of the condition variable sequencing utility (`condvar.c`)

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>

char message[32];
int ready = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t changed = PTHREAD_COND_INITIALIZER;

void *writer(void *arg) {
    (void)arg;
    sleep(1);
    pthread_mutex_lock(&lock);
    strcpy(message, "hello");
    ready = 1;
    printf("A: message written\n");
    pthread_cond_signal(&changed);
    pthread_mutex_unlock(&lock);
    return NULL;
}

void *reader(void *arg) {
    (void)arg;
    pthread_mutex_lock(&lock);
    while (!ready) {
        printf("B: not ready, I sleep\n");
        pthread_cond_wait(&changed, &lock);
    }
    printf("B: I read \"%s\"\n", message);
    pthread_mutex_unlock(&lock);
    return NULL;
}

int main(void) {
    pthread_t a, b;
    pthread_create(&b, NULL, reader, NULL);
    pthread_create(&a, NULL, writer, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    return 0;
}
```

### A.3 Source code of the readers and writers synchronization utility (`readers.c`)

```
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define READERS 3
#define WRITERS 2

int value = 0;
int readers = 0;
sem_t rlock;
sem_t room;

```

```c
void *reader(void *arg) {
    long id = (long)arg;
    sem_wait(&rlock);
    if (++readers == 1)
        sem_wait(&room);
    sem_post(&rlock);

    printf("reader %ld reads %d (readers inside: %d)\n", id, value, readers);
    usleep(200000);

    sem_wait(&rlock);
    if (--readers == 0)
        sem_post(&room);
    sem_post(&rlock);
    return NULL;
}

void *writer(void *arg) {
    long id = (long)arg;
    sem_wait(&room);
    value += 10;
    printf("writer %ld writes %d (alone)\n", id, value);
    usleep(200000);
    sem_post(&room);
    return NULL;
}

int main(void) {
    sem_init(&rlock, 0, 1);
    sem_init(&room, 0, 1);

    pthread_t r[READERS], w[WRITERS];
    pthread_create(&w[0], NULL, writer, (void *)1L);
    for (long i = 0; i < READERS; i++)
        pthread_create(&r[i], NULL, reader, (void *)(i + 1));
    pthread_create(&w[1], NULL, writer, (void *)2L);

    for (int i = 0; i < READERS; i++) pthread_join(r[i], NULL);
    for (int i = 0; i < WRITERS; i++) pthread_join(w[i], NULL);

    sem_destroy(&rlock);
    sem_destroy(&room);
    return 0;
}
```

### A.4 Source code of the deadlock demonstration and resolution utility (`deadlock.c`)

```c
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

pthread_mutex_t A = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t B = PTHREAD_MUTEX_INITIALIZER;

void *one(void *arg) {
    (void)arg;
    pthread_mutex_lock(&A);
    printf("one: I have A, I want B\n");
    sleep(1);
    pthread_mutex_lock(&B);
    printf("one: I have both\n");
    pthread_mutex_unlock(&B);
    pthread_mutex_unlock(&A);
    return NULL;
}
```

```c

void *two(void *arg) {
    (void)arg;
    /* Fixed: lock A first, then B, adhering to globally consistent acquisition ordering
       */
    pthread_mutex_lock(&A);
    printf("two: I have A, I want B\n");
    sleep(1);
    pthread_mutex_lock(&B);
    printf("two: I have both\n");
    pthread_mutex_unlock(&B);
    pthread_mutex_unlock(&A);
    return NULL;
}

int main(void) {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, one, NULL);
    pthread_create(&t2, NULL, two, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("done\n");
    return 0;
}
```

### A.5 Build automation script (`Makefile`)

```
CC = gcc
CFLAGS = -Wall -pthread
SHELL = /bin/bash

PROGS = prodcons condvar readers deadlock

all: $(PROGS)

prodcons: prodcons.c
    $(CC) $(CFLAGS) -o $@ $<

condvar: condvar.c
    $(CC) $(CFLAGS) -o $@ $<

readers: readers.c
    $(CC) $(CFLAGS) -o $@ $<

deadlock: deadlock.c
    $(CC) $(CFLAGS) -o $@ $<

test: all
    @echo "--- Testing prodcons ---"
    ./prodcons
    @echo "--- Testing condvar ---"
    ./condvar
    @echo "--- Testing readers ---"
    ./readers
    @echo "--- Testing deadlock (fixed) ---"
    ./deadlock

clean:
    rm -f $(PROGS)

.PHONY: all test clean
```